Computer Architecture
CS 622 / EE 520
Fall 2026

Lecture 16

Shahid Masud

Topics

- Memory Hierarchy in Computers
  - Technology and Cost
- Locality of Reference
- Cache Organization
  - Hierarchy
  - Tags
  - Mapping Techniques
- Cache Operation
  - Replacement
  - Performance

Midterm Exam
Next Lecture

Further Readings

Reading Notes

Readings:
Appendix B of course textbook

Slides adapted from:
https://www.cs.nthu.edu.tw/~king/courses/cs5100/2014/N04-Cache-1.pdf

Memory Access Performance

Memory Performance Gap

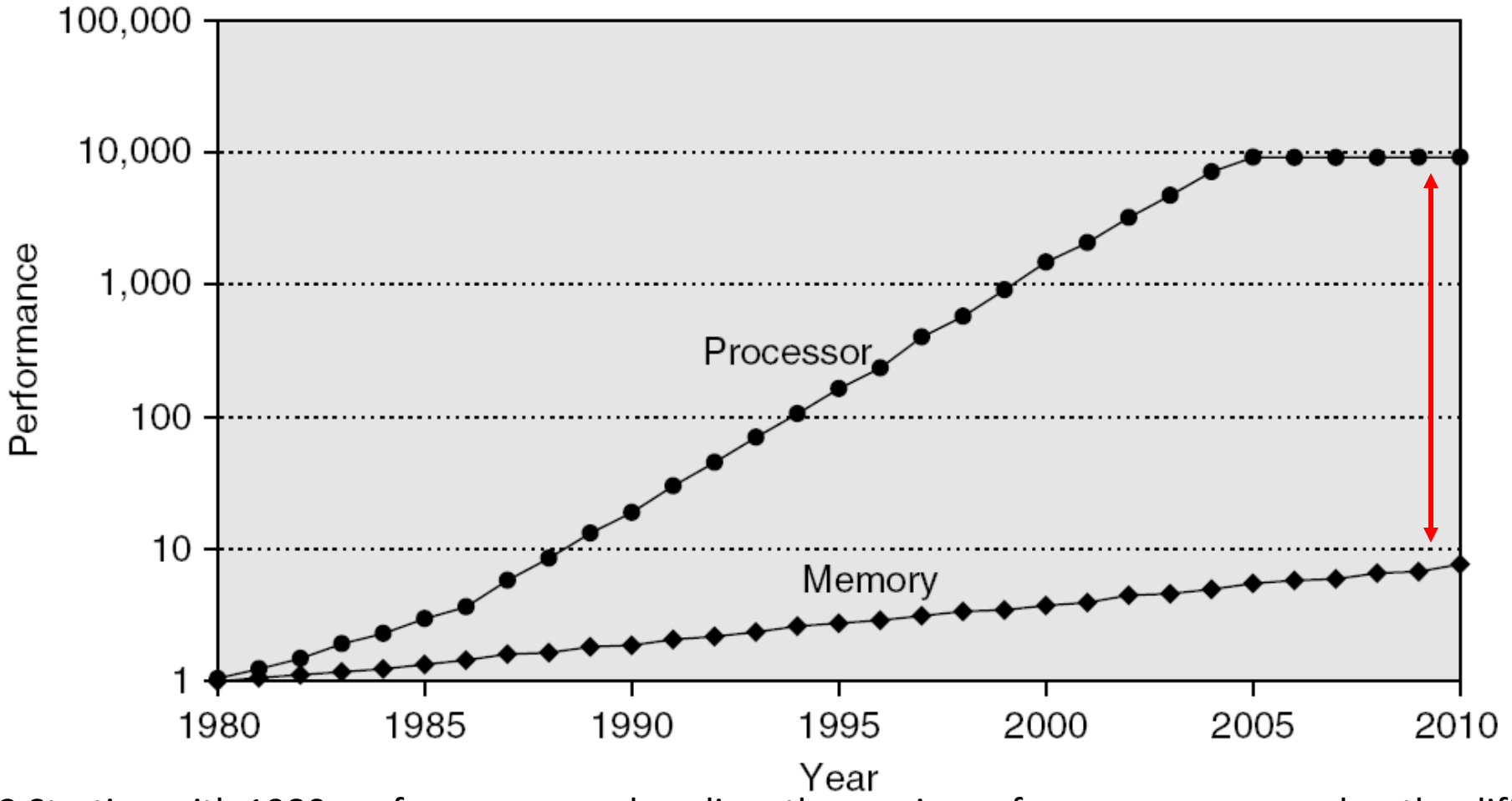


Fig. 2.2 Figure 2.2 Starting with 1980 performance as a baseline, the gap in performance, measured as the difference in the time between processor memory requests (for a single processor or core) and the latency of a DRAM access, is plotted over time. Note that the vertical axis must be on a logarithmic scale to record the size of the processor–DRAM performance gap. The memory baseline is 64 KB DRAM in 1980, with a 1.07 per year performance improvement in Latency.

Memory Access Time Comparison

Features	SRAM	DRAM	PCM	NAND FLASH
Memory Cell Factor(F^2)	50-200	6	4-10	2-5
Read Time (ns)	<1	30	10-100	10^4
Write /Erase Time (ns)	<1	50	100-300	10^5
Number of Rewrites	10^{16}	10^{16}	$10^8 - 10^{15}$	$10^4 - 10^5$
Dyn. Power	Low	Medium	Medium read High write	Very high read Even high write
Leak. Power	High	Medium	Low	Low
Data Retention	N	N	Y	Y

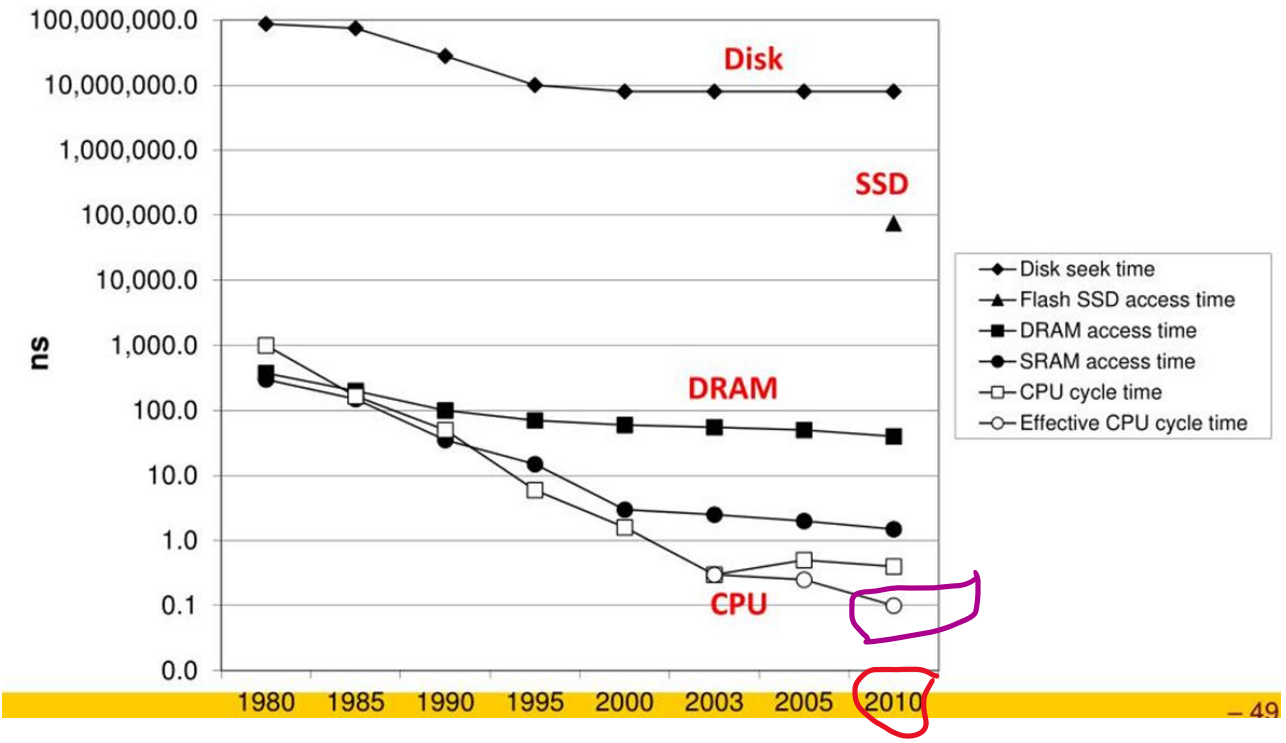
Memory Access Technology Comparison

	SRAM	DRAM	Flash (NOR)	Flash (NAND)	FeRAM	MRAM	PRAM	RRAM	STT-RAM
Non-volatile	No	No	Yes	Yes	Yes	Yes	Yes	Yes	Yes
Cell size (F^2)	50-120	6-10	10	5	15-34	16-40	6-12	6-10	6-20
Read time (ns)	1-100	30	10	50	20-80	3-20	20-50	10-50	2-20
Write / Erase time (ns)	1-100	15	1 μs / 10 ms	1 ms / 0.1 ms	50 / 50	3-20	60 / 120	10-50	2-20
Endurance	10^{16}	10^{16}	10^{5}	10^{5}	10^{12}	$>10^{15}$	10^{8}	10^{8}	$>10^{15}$
Write power	Low	Low	Very high	Very high	Low	High	High	Low	Low
Other power consumption	Current leakage	Refresh current	None	None	None	None	None	None	None
High voltage required	No	3 V	6-8 V	16-20 V	2-3 V	3 V	1.5-3 V	1.5-3 V	<1.5 V
	Existing products						*Prototype*		

CPU Memory Gap

The CPU-Memory Gap

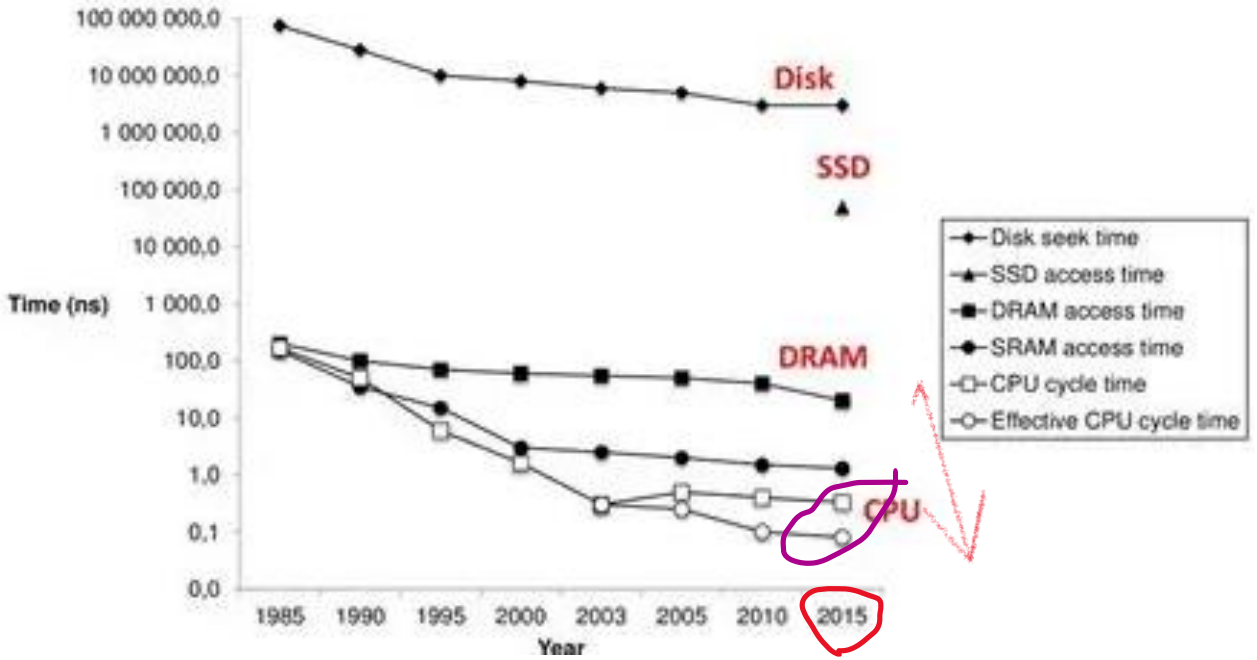
The gap widens between DRAM, disk, and CPU speeds.



– 49

The CPU-Memory Gap

The gap widens between DRAM, disk, and CPU speeds.



Variations in Memory Performance



Level	Memory Technology	Size	Typical Access Time	Cost per Mbyte
Registers	D Flip-Flops	64 32-bit	2 -3 ns	N/A
L1 Cache (on chip)	SRAM	16 Kbytes	5 - 25 ns	$100 - $250
L2Cache (off chip)	SRAM	256 Kbytes	5 - 25 ns	$100 - $250
Main Memory	DRAM	64 Mbytes	60 - 120 ns	$5 - $10
Secondary Storage	Magnetic Disk	2 Gbytes	10 - 20 ms	$0.10-$0.20

Memory Hierarchy

Memory Hierarchy Motivation

- Memory hierarchy design becomes more crucial with recent multi-core processors:
- Aggregate peak bandwidth grows with # cores:
  - Intel Core i7 generates two references per core per clock
  - Four cores and 3.2 GHz clock:
    25.6 billion 64-bit data references/second + 12.8 billion 128-bit instruction references = 409.6 GB/s!
  - DRAM bandwidth is only 6% of this (25 GB/s)
  - Requires:
    - Multi-port, pipelined caches
    - Two levels of cache per core
    - Shared third-level cache on chip

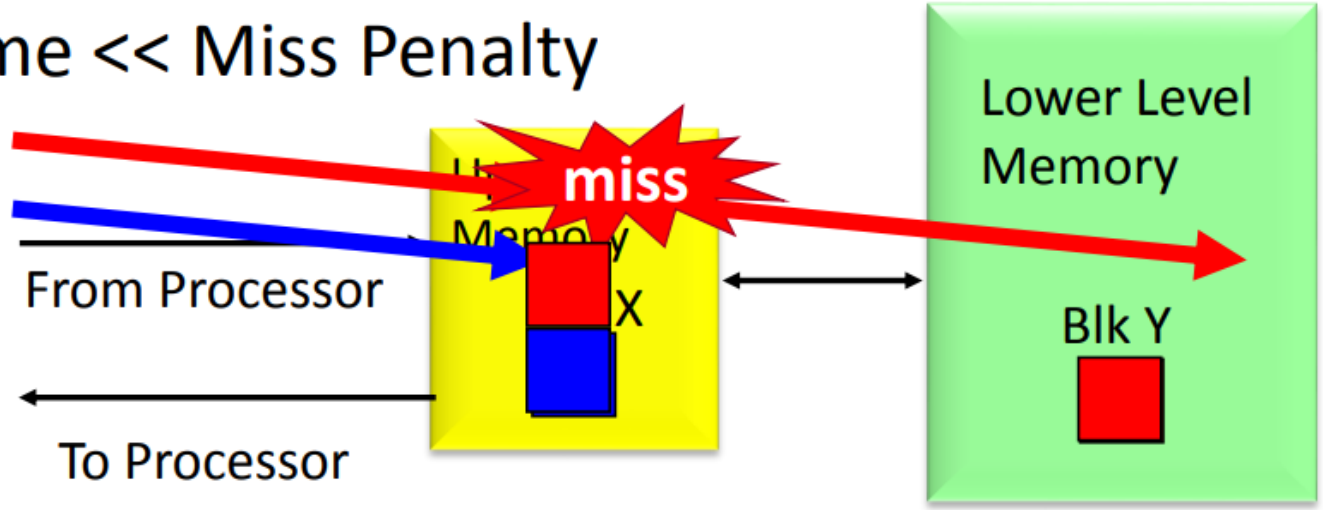
Thinking behind Memory Hierarchy Design

- Entire addressable memory space available in largest, slowest memory
- Incrementally smaller and faster memories, each containing a subset of the memory below it, proceed in steps up toward the processor

Memory Hierarchy Basics

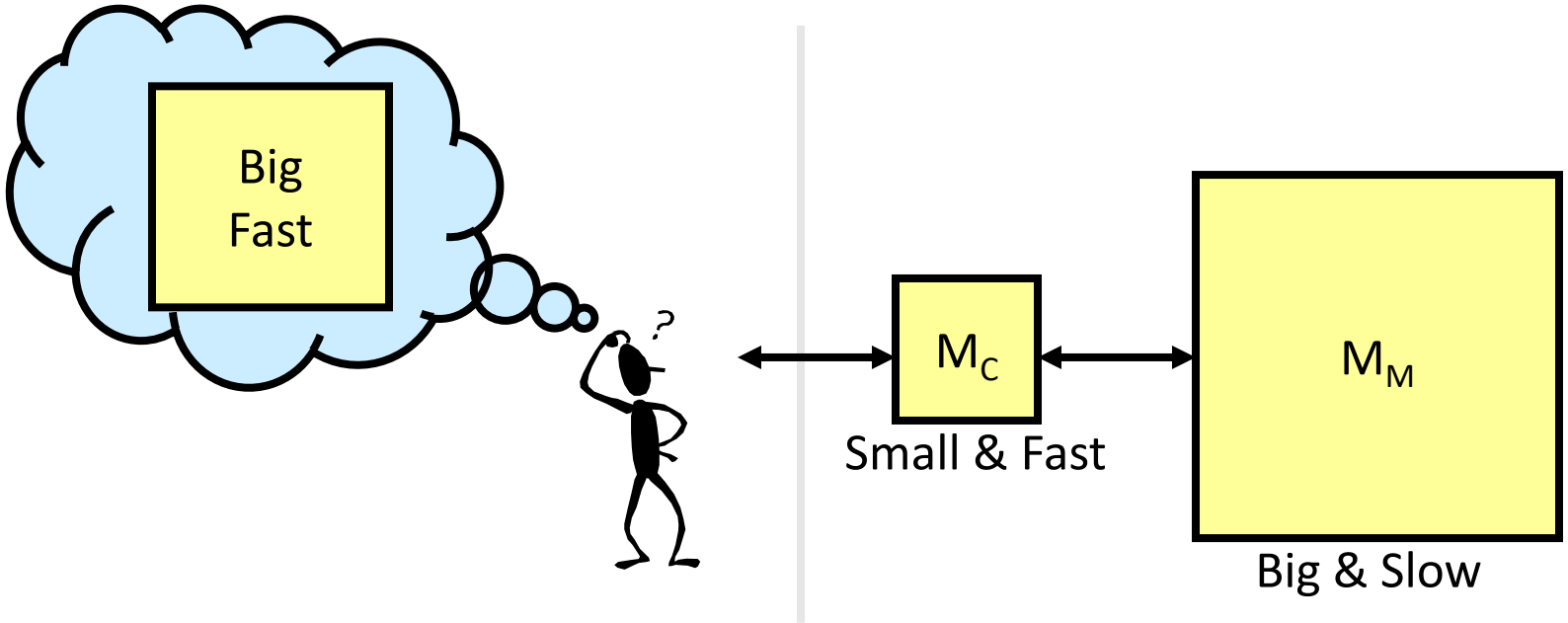
- *Hit*: data appears in some *block* (*page*) in upper level
  - Hit rate: % of memory access found in upper level
  - Hit time: time to access upper level
- *Miss*: data not in upper level
  - Miss rate = 1 - (hit rate)
  - Miss penalty: time to replace a block in upper level + time to deliver the block to the processor
- Hit Time << Miss Penalty

Hierarchy



Motivation for Memory Hierarchy

- Programmers want unlimited amounts of memory with low latency
  - But fast memory more expensive than slower memory
- Solution: small fast memory + big slow memory
  = Looks like a big fast memory



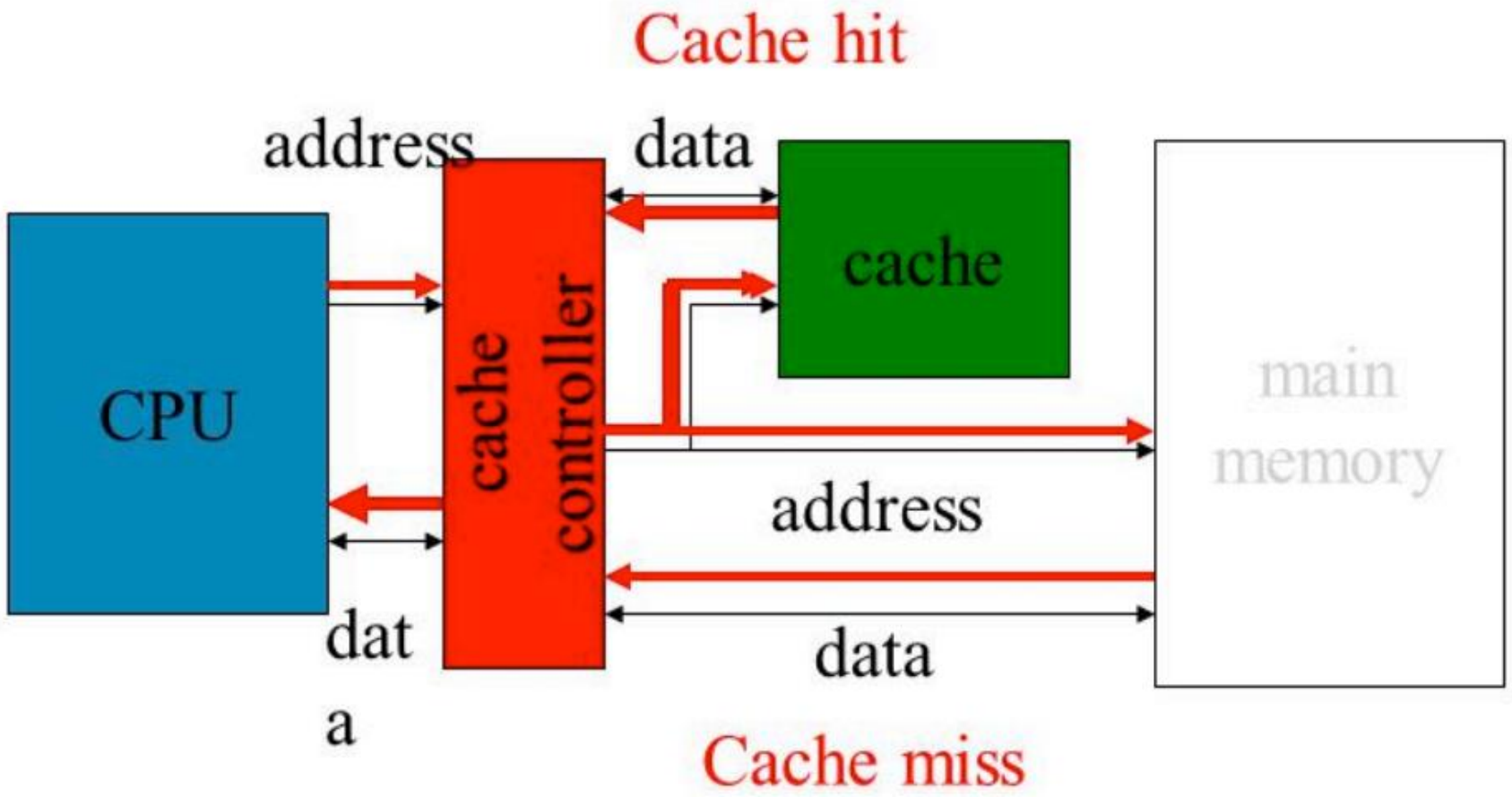
Memory Hierarch Access Time

Level	1	2	3	4
Name	Registers	Cache	Main memory	Disk storage
Typical size	<4 KiB	32 KiB to 8 MiB	<1 TB	>1 TB
Implementation technology	Custom memory with multiple ports, CMOS	On-chip CMOS SRAM	CMOS DRAM	Magnetic disk or FLASH
Access time (ns)	0.1–0.2	0.5–10	30–150	5,000,000
Bandwidth (MiB/sec)	1,000,000–10,000,000	20,000–50,000	10,000–30,000	100–1000
Managed by	Compiler	Hardware	Operating system	Operating system
Backed by	Cache	Main memory	Disk or FLASH	Other disks and DVD

Figure B.1 The typical levels in the hierarchy slow down and get larger as we move away from the processor for a large workstation or small server. Embedded computers might have no disk storage and much smaller memories and caches. Increasingly, FLASH is replacing magnetic disks, at least for first level file storage. The access times increase as we move to lower levels of the hierarchy, which makes it feasible to manage the transfer less responsively. The implementation technology shows the typical technology used for these functions. The access time is given in nanoseconds for typical values in 2017; these times will decrease over time. Bandwidth is given in megabytes per second between levels in the memory hierarchy. Bandwidth for disk/FLASH storage includes both the media and the buffered interfaces.

Cache Organization and Placement

Cache Placement and Memory Access



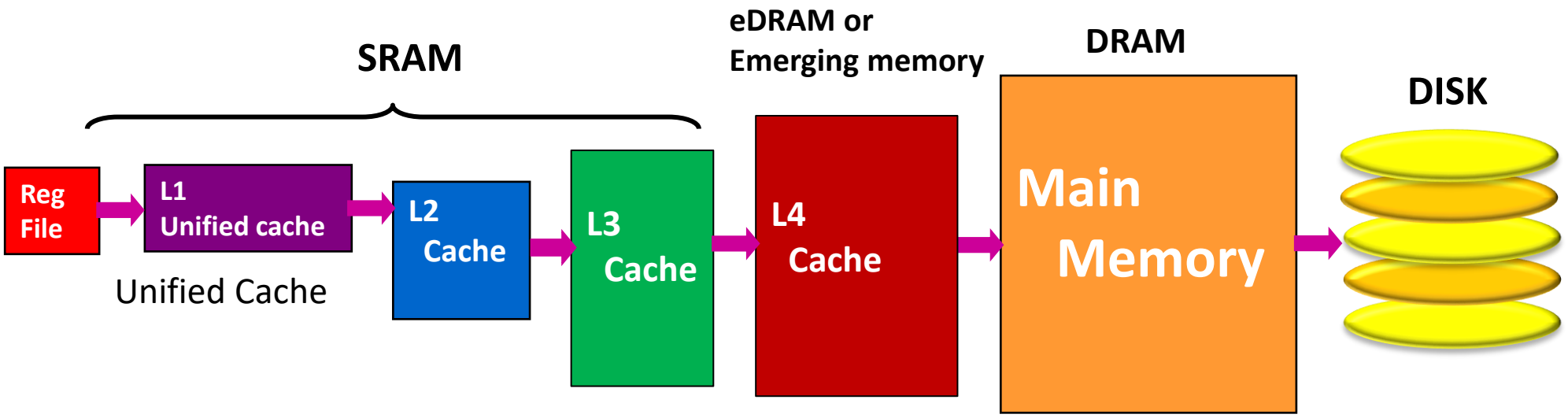
What goes in a Cache?

Cache may hold certain types of information

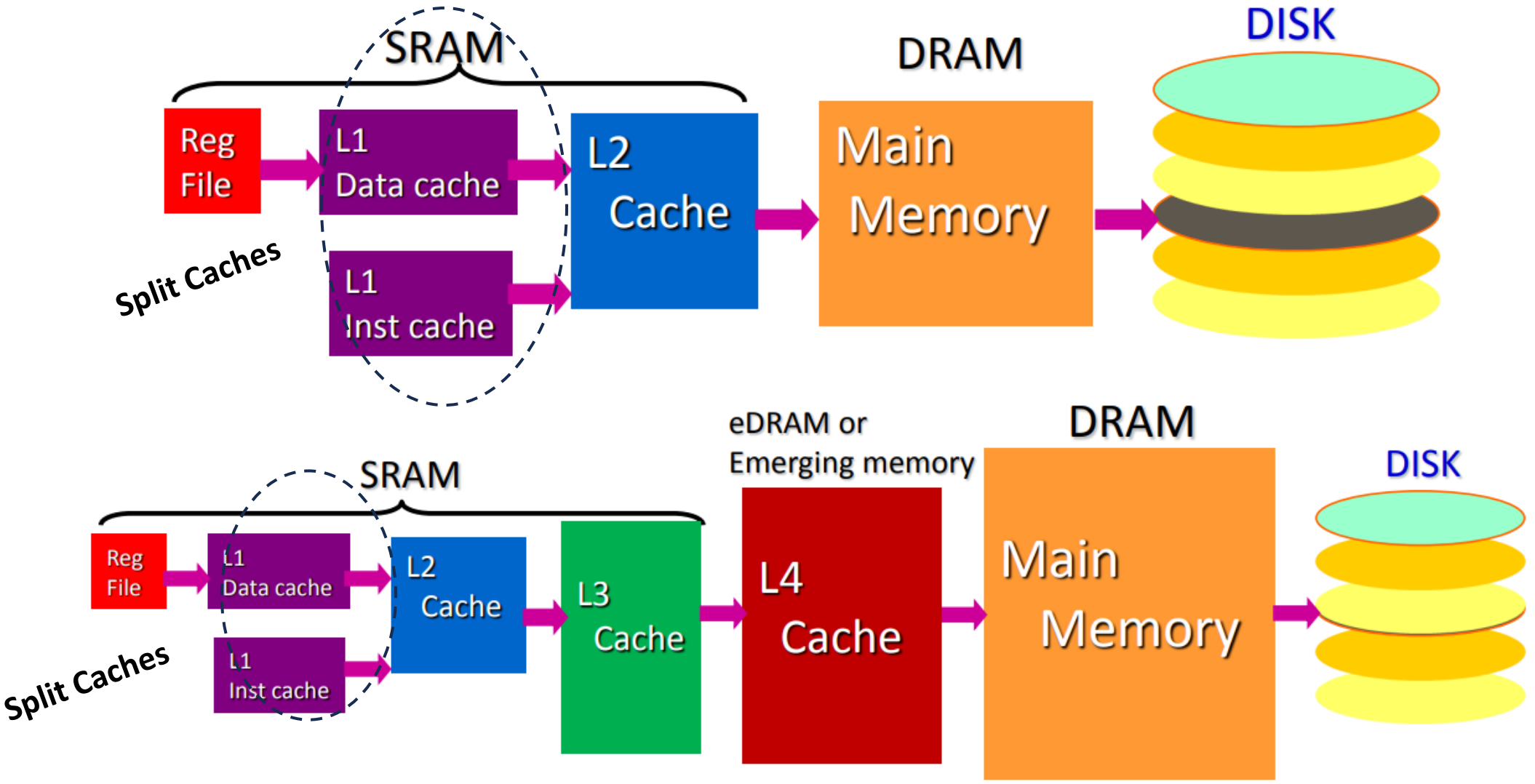
- Instruction cache:
  - Read-only (mostly), very good spatial locality
    - sequential instruction stream and loops
    - but sometimes lots of subroutine calls
- Data cache:
  - read/write with different spatial locality pattern
- Stack cache:
  - captures a window of memory addresses including the top-of-stack
- Unified cache:
  - holds everything - data + instructions

Memory Hierarchy Design 1

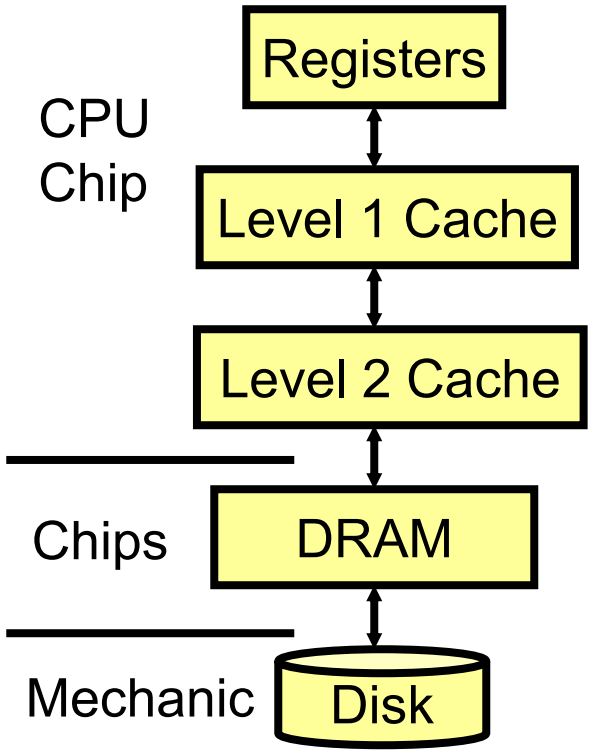
- Entire addressable memory space available in largest, slowest memory
- Incrementally smaller and faster memories, each containing a subset of the memory below it, proceed in steps up toward the processor



Another possible Memory Hierarchy

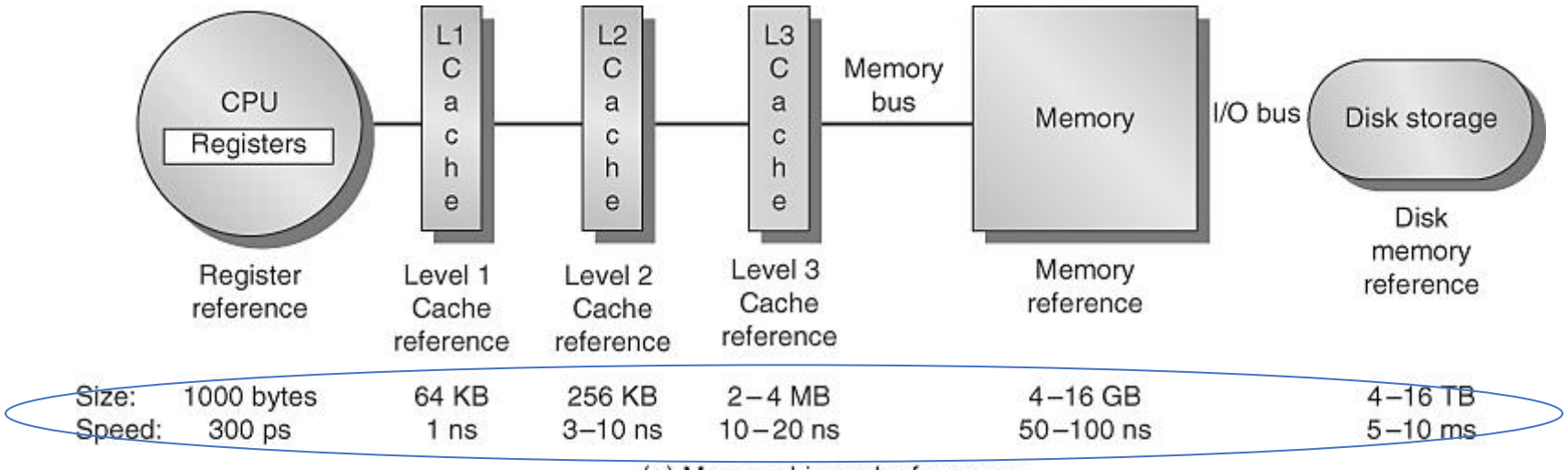


Model of Memory Hierarchy

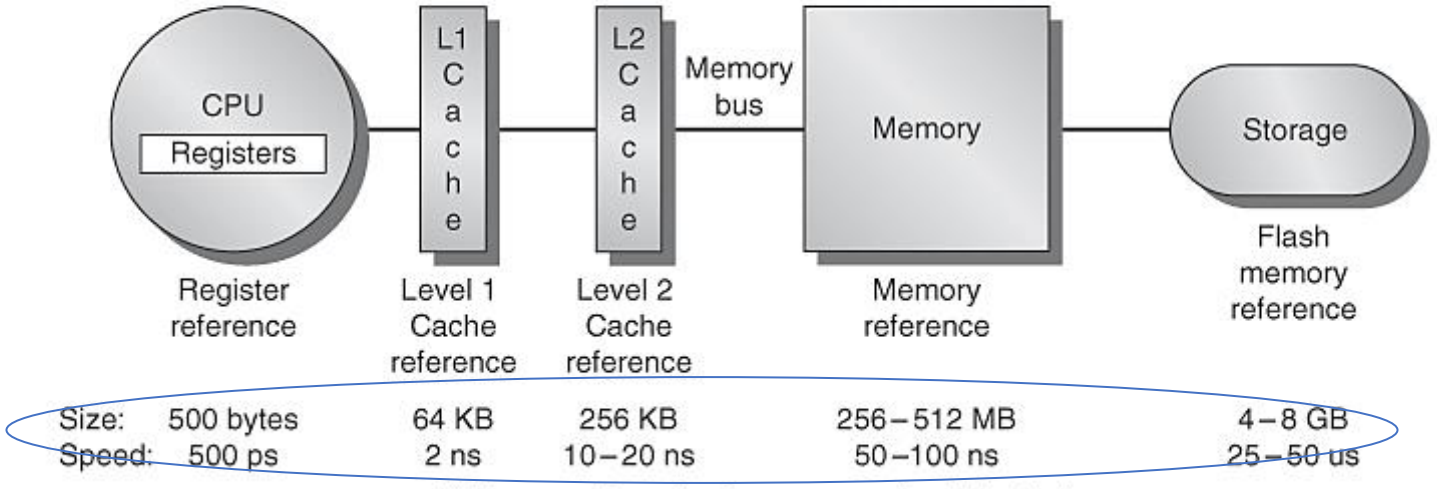


	Cycle	Words/cycle	Management	
Registers	1	3-10	Compiler	
Level 1 Cache	1-3	1-2	Hardware	
Level 2 Cache	5-10	1	Hardware OS	
DRAM	30-100	0.5	OS	
Disk	10^6-10^7	0.01	?	Hardware + Software

Model of Memory Hierarchy



(a) Memory hierarchy for server



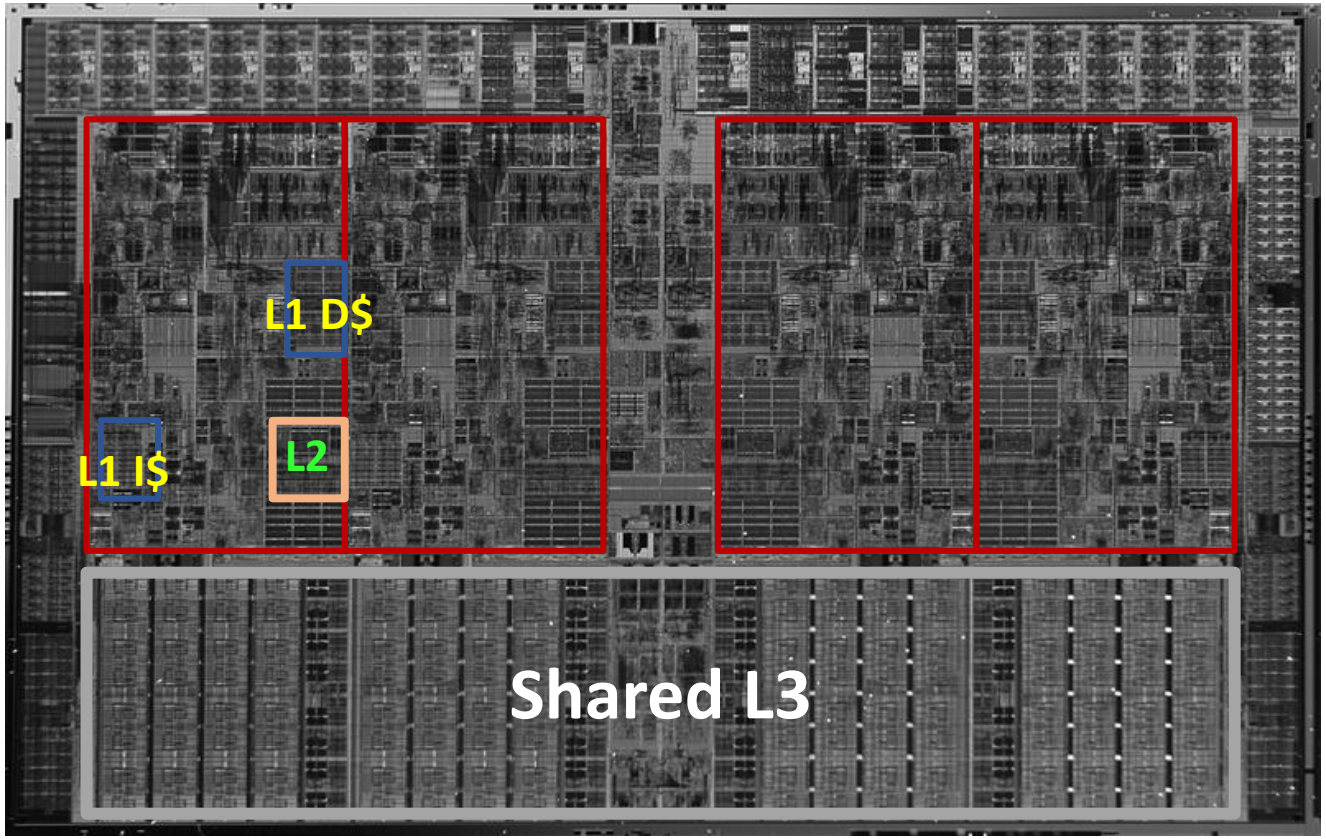
(b) Memory hierarchy for a personal mobile device

Fig. 2.1 The levels in a typical memory hierarchy in a server computer shown on top (a) and in a personal mobile device (PMD) on the bottom (b). As we move farther away from the processor, the memory in the level below becomes slower and larger.

Cache Power Consumption

- High-end μPs have >10 MB on-chip cache
  - Consumes large amount of area and power budget

Intel Core i7 Cache example



Principle of Locality

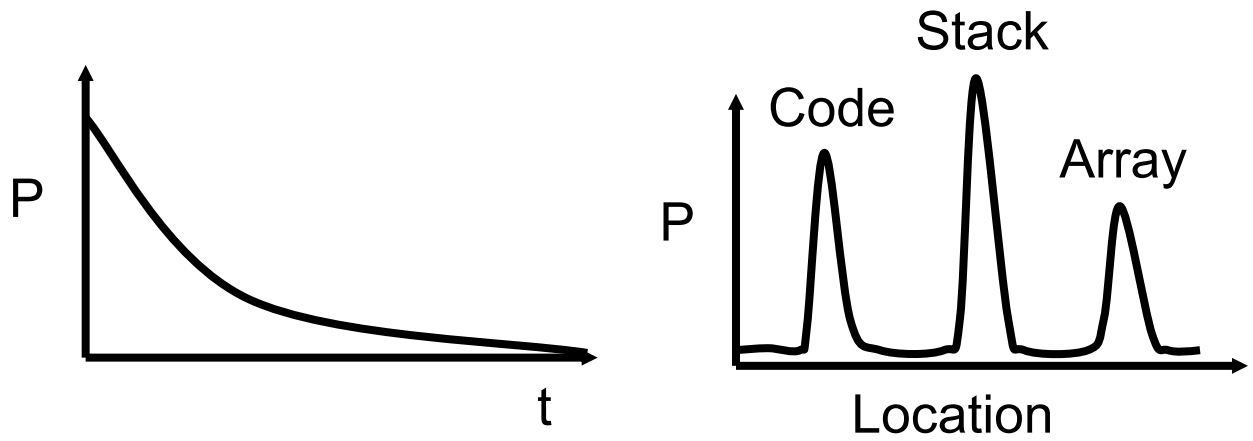
Why Hierarchy Works?

- The Principle of Locality:
  - Program access a relatively small portion of the address space at any instant of time.



Underlying Principle: Locality of Reference

- Locality of reference:
  - Program access a relatively small portion of the address space at any instant of time → a program property
  - Temporal locality: If an item is referenced, it will tend to be referenced again soon (e.g., loops, reuse)
  - Spatial locality: If an item is referenced, items whose addresses are close by tend to be referenced soon (e.g., straightline code, array access)



Locality Explained

- Programs tend to reuse data and instructions near those they have used recently, or that were recently referenced themselves
- **Spatial locality:** Items with nearby addresses tend to be referenced close together in time
- **Temporal locality:** Recently referenced items are likely to be referenced in the near future

```
sum = 0;
for(i=0; i<n; i++)
  sum += a[i];
return sum;
```

- **Data**
  - Reference array elements in succession (stride-1 reference pattern): Spatial Locality
  - Reference sum each iteration: Temporal Locality
- **Instructions**
  - Reference instructions in sequence: Spatial Locality
  - Cycle through loop repeatedly: Temporal Locality

Locality Example 1

- **Claim: Being able to look at code and *get qualitative sense* of its locality is key skill for professional programmer**

- Question: Does this function have good locality?

```
int sumarrayrows(int a[M][N]){
    int i, j, sum = 0;

    for (i = 0; i < M; i++)
        for (j = 0; j < N; j++)
            sum += a[i][j];
    return sum;
}
```



Locality Example 2

- Question: Does this function have good locality?

```
int sumarraycols(int a[M][N]){
    int i, j, sum = 0;

    for (j = 0; j < N; j++)
        for (i = 0; i < M; i++)
            sum += a[i][j];
    return sum;
}
```



Source of Locality

- Temporal locality
  - Code within a loop
  - Same instructions fetched repeatedly
- Spatial locality
  - Data arrays
  - Local variables in stack
  - Data allocated in chunks (contiguous bytes)

```
for (i=0; i<N; i++) {
    A[i] = B[i] + C[i] * a;
}
```

Writing Cache Friendly Code

- Repeated references to variables are good (temporal locality)
- Stride-1 reference patterns are good (spatial locality)
- Examples:
  - cold cache, 4-byte words, 4-word cache blocks

```
int sumarrayrows(int a[M][N])
{
    int i, j, sum = 0;

    for (i = 0; i < M; i++)
        for (j = 0; j < N; j++)
            sum += a[i][j];
    return sum;
}
```

Miss rate = 1/4 = 25%

```
int sumarraycols(int a[M][N])
{
    int i, j, sum = 0;

    for (j = 0; j < N; j++)
        for (i = 0; i < M; i++)
            sum += a[i][j];
    return sum;
}
```

Miss rate = 100%

Program Behavior Matters

- Locality depends on type of program
- Some programs ‘behave’ well
  - Small loop operating on data on stack (towers of hanoi)
- Some programs don’t
  - Frequent calls to nearly random subroutines
  - Traversal of large, sparse data set
    - Pointer-based data structures
    - Essentially random data references with no reuse
  - Traversal of large sequential data
    - Streaming and multimedia data

Basics of Cache

- When a word is not found in the cache, a *miss* occurs:
  - Fetch word from lower level in hierarchy, requiring a higher latency reference
  - Lower level may be another cache or the main memory
  - Also fetch the other words contained within the *block*

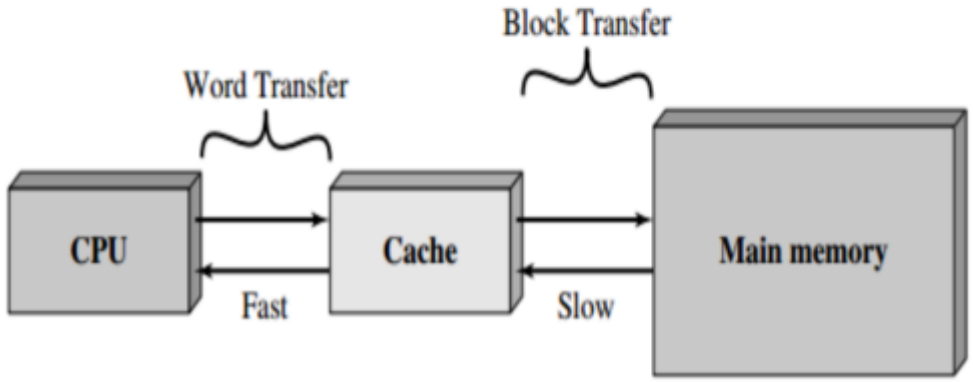
  - Place block into cache in any location within its *set*, determined by address
    - Block address MOD number of sets

Cache Design and Access

Considerations for Cache Design

- Q1: Where can a block be placed in the upper level?
  - Block placement
- Q2: How is a block found if it is in the upper level?
  - Block identification
- Q3: Which block should be replaced on a miss?
  - Block replacement
- Q4: What happens on a write?
  - Write strategy

Read hit



(a) Single cache

Cache and Memory Relation

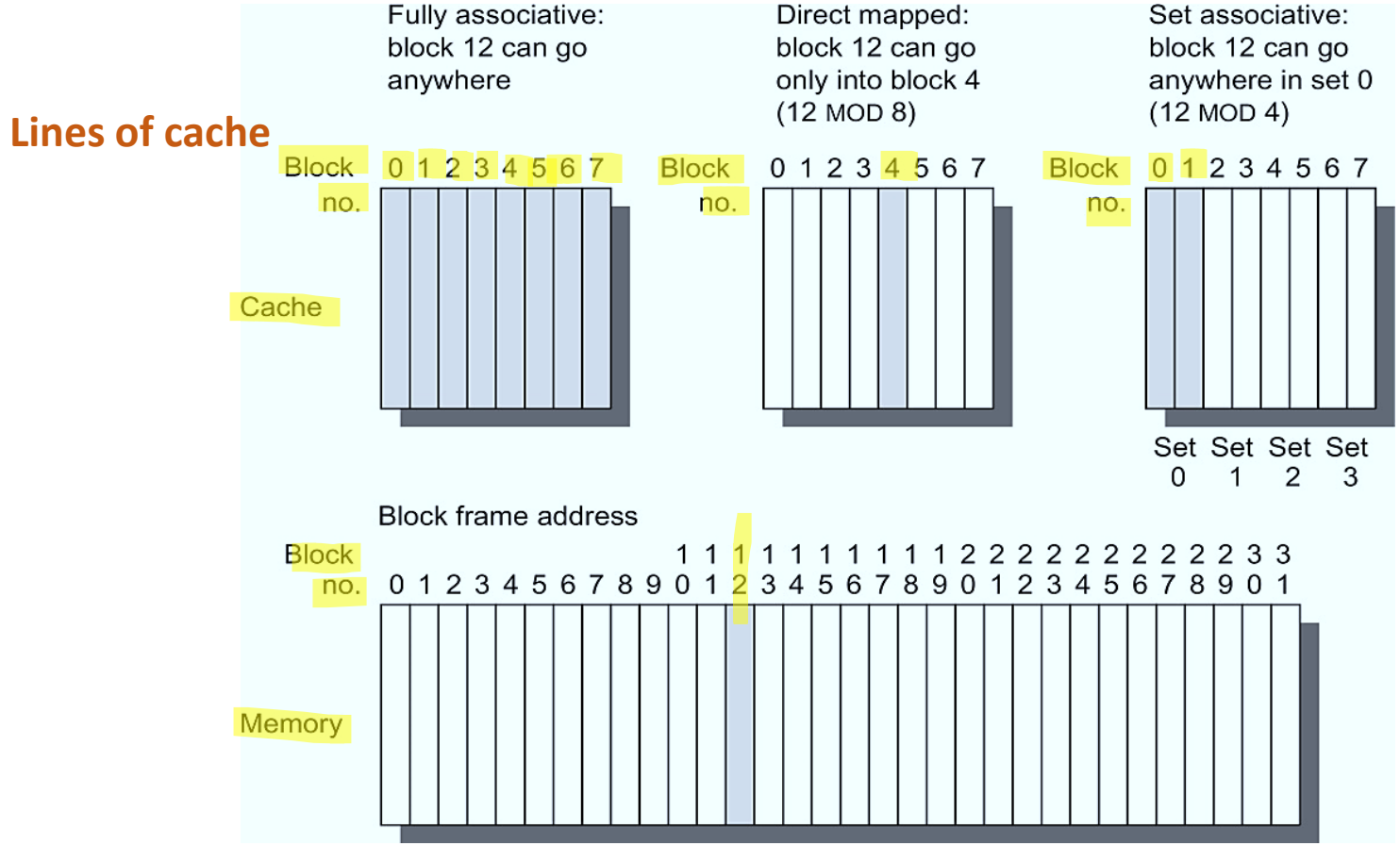


Figure B.2 This example cache has eight block frames and memory has 32 blocks. The three options for caches are shown left to right. In fully associative, block 12 from the lower level can go into any of the eight block frames of the cache. With direct mapped, block 12 can only be placed into block frame 4 (12 modulo 8). Set associative, which has some of both features, allows the block to be placed anywhere in set 0 (12 modulo 4). With two blocks per set, this means block 12 can be placed either in block 0 or in block 1 of the cache. Real caches contain thousands of block frames, and real memories contain millions of blocks. The set associative organization has four sets with two blocks per set, called *two-way set associative*. Assume that there is nothing in the cache and that the block address in question identifies lower-level block 12.

Parts of Cache Address

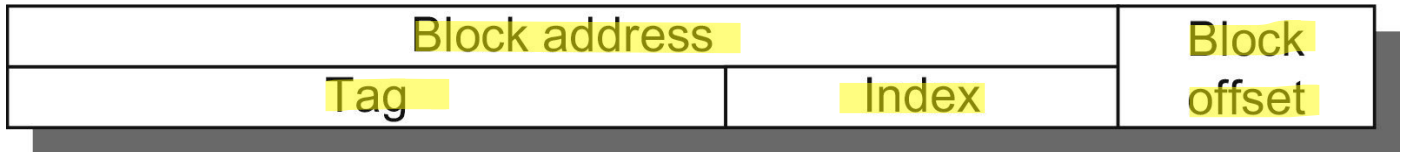
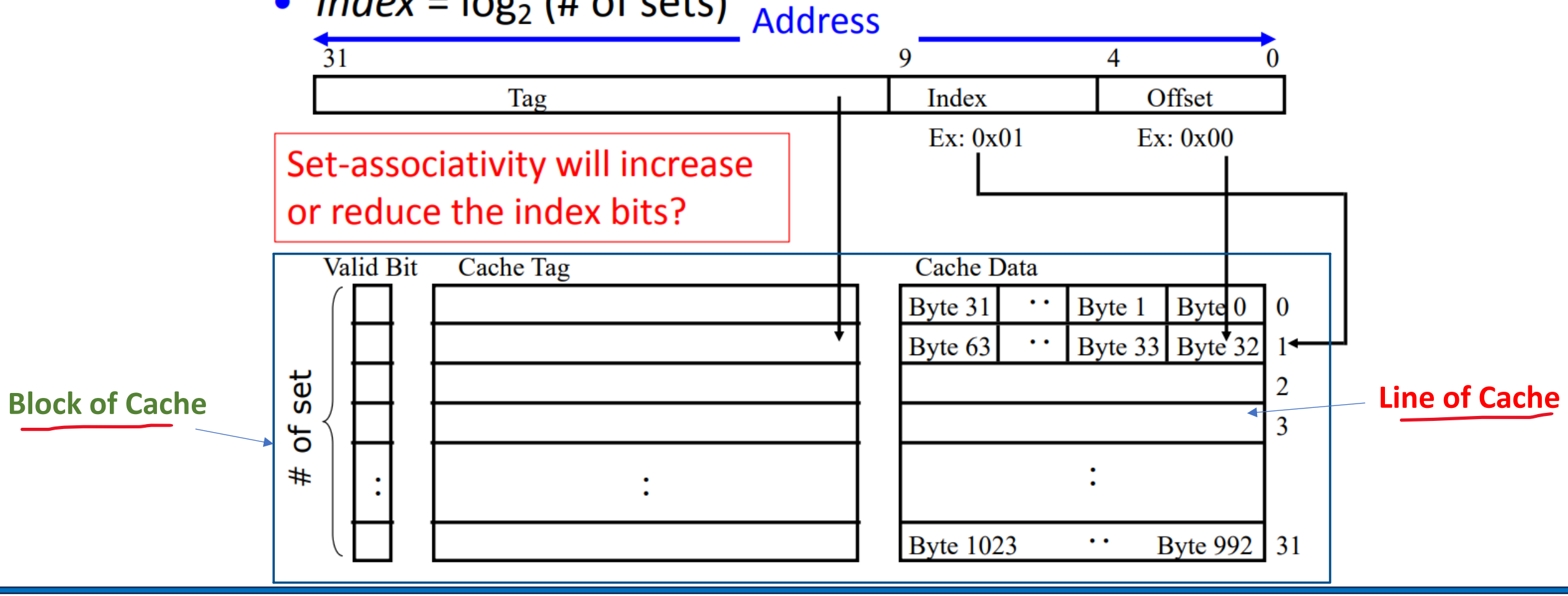


Figure B.3 The three portions of an address in a set associative or direct-mapped cache. The tag is used to check all the blocks in the set, and the index is used to select the set. The block offset is the address of the desired data within the block. Fully associative caches have no index field.

Example of Direct Mapped DM Cache

Example: 1KB DM Cache, 32-byte Blocks

- The lowest M bits are the *offset* (block size = 2^M= 32)
- *Index* = $\log_2$ (# of sets)



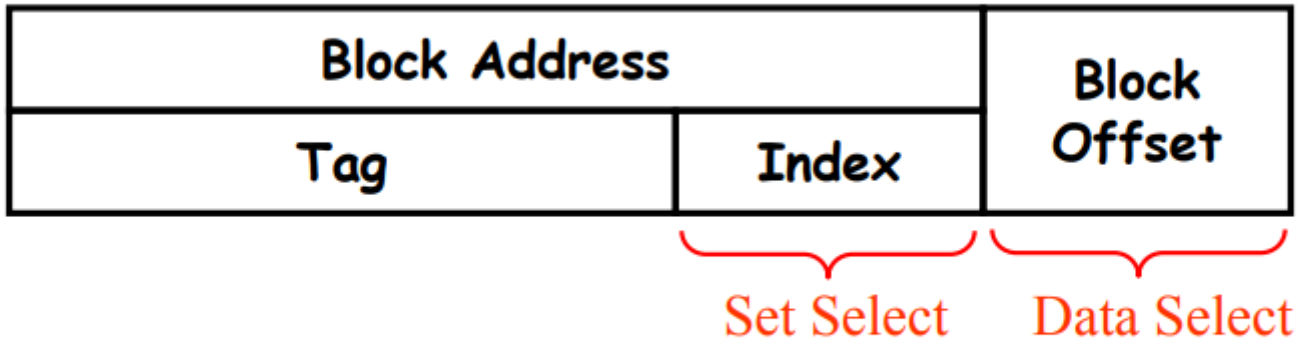
Identifying Data Location in a Cache

Q2: Block Identification

Cache Block

	Cache Line		

- Tag on each block
  - No need to check index or block offset
- Increasing associativity shrinks index, expands tag



$$\text{Cache size} = \text{Associativity} \times 2^{\text{index_size}} \times 2^{\text{offest_size}}$$

Studying Replacement Policies

	Associativity								
	Two-way			Four-way			Eight-way		
Size	LRU	Random	FIFO	LRU	Random	FIFO	LRU	Random	FIFO
16 KiB	114.1	117.3	115.5	111.7	115.1	113.3	109.0	111.8	110.4
64 KiB	103.4	104.3	103.9	102.4	102.3	103.1	99.7	100.5	100.3
256 KiB	92.2	92.1	92.5	92.1	92.1	92.5	92.1	92.1	92.5

Figure B.4 Data cache misses per 1000 instructions comparing least recently used, random, and first in, first out replacement for several sizes and associativities. There is little difference between LRU and random for the largest size cache, with LRU outperforming the others for smaller caches. FIFO generally outperforms random in the smaller cache sizes. These data were collected for a block size of 64 bytes for the Alpha architecture using 10 SPEC2000 benchmarks. Five are from SPECint2000 (gap, gcc, gzip, mcf, and perl) and five are from SPECfp2000 (applu, art, equake, lucas, and swim). We will use this computer and these benchmarks in most figures in this appendix.

Organization of Cache in Opteron Processor

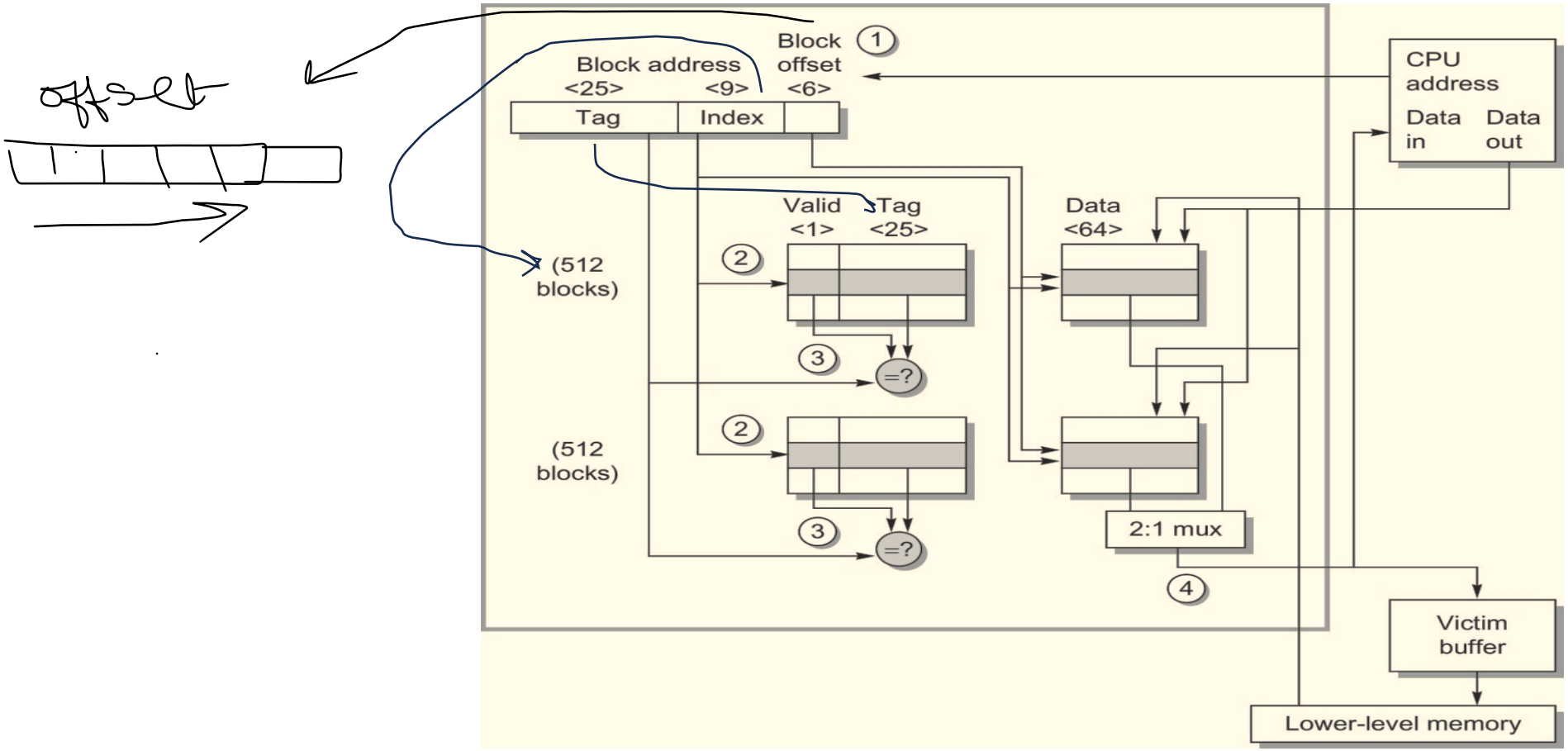


Figure B.5 The organization of the data cache in the Opteron microprocessor. The 64 KiB cache is two-way set associative with 64-byte blocks. The 9-bit index selects among 512 sets. The four steps of a read hit, shown as circled numbers in order of occurrence, label this organization. Three bits of the block offset join the index to supply the RAM address to select the proper 8 bytes. Thus, the cache holds two groups of 4096 64-bit words, with each group containing half of the 512 sets. Although not exercised in this example, the line from lower-level memory to the cache is used on a miss to load the cache. The size of address leaving the processor is 40 bits because it is a physical address and not a virtual address. Figure B.24 on page B-47 explains how the Opteron maps from virtual to physical for a cache access.

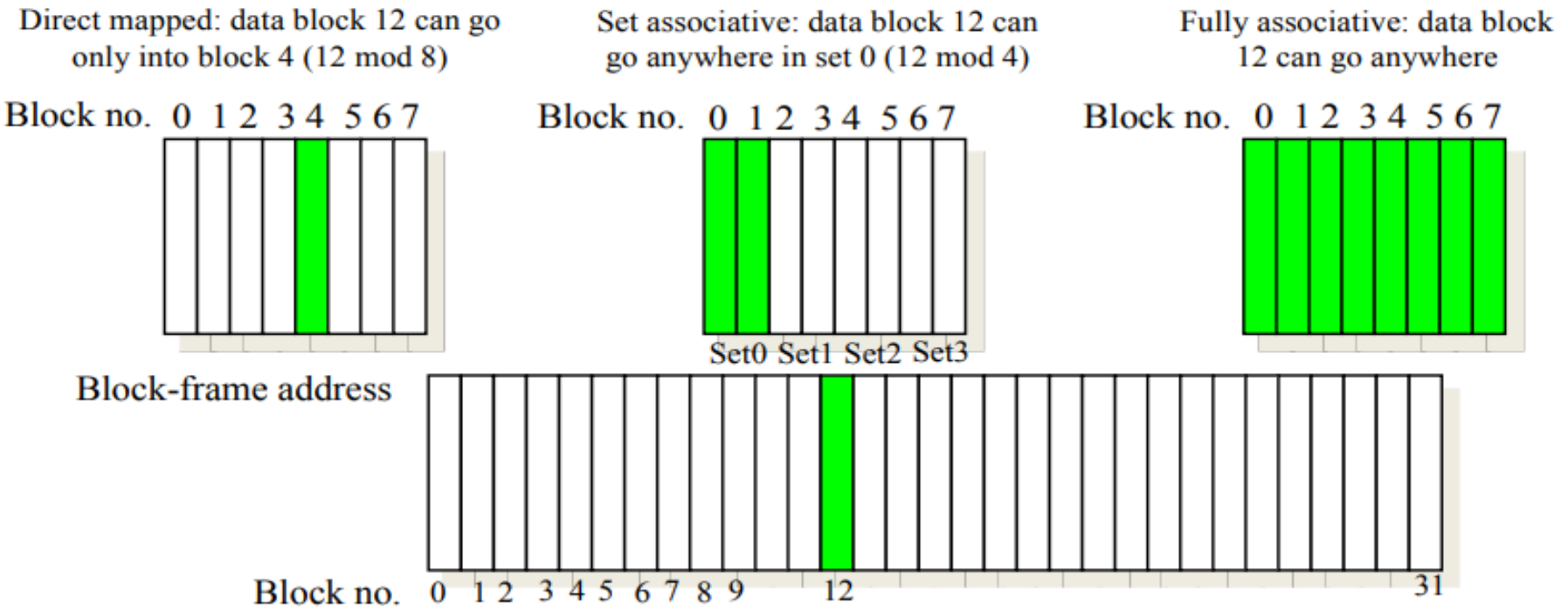
Study Cache Miss Behavior

Size (KiB)	Instruction cache	Data cache	Unified cache
8	8.16	44.0	63.0
16	3.82	40.9	51.0
32	1.36	38.4	43.3
64	0.61	36.9	39.4
128	0.30	35.3	36.2
256	0.02	32.6	32.9

Figure B.6 Miss per 1000 instructions for instruction, data, and unified caches of different sizes. The percentage of instruction references is about 74%. The data are for two-way associative caches with 64-byte blocks for the same computer and benchmarks as Figure B.4.

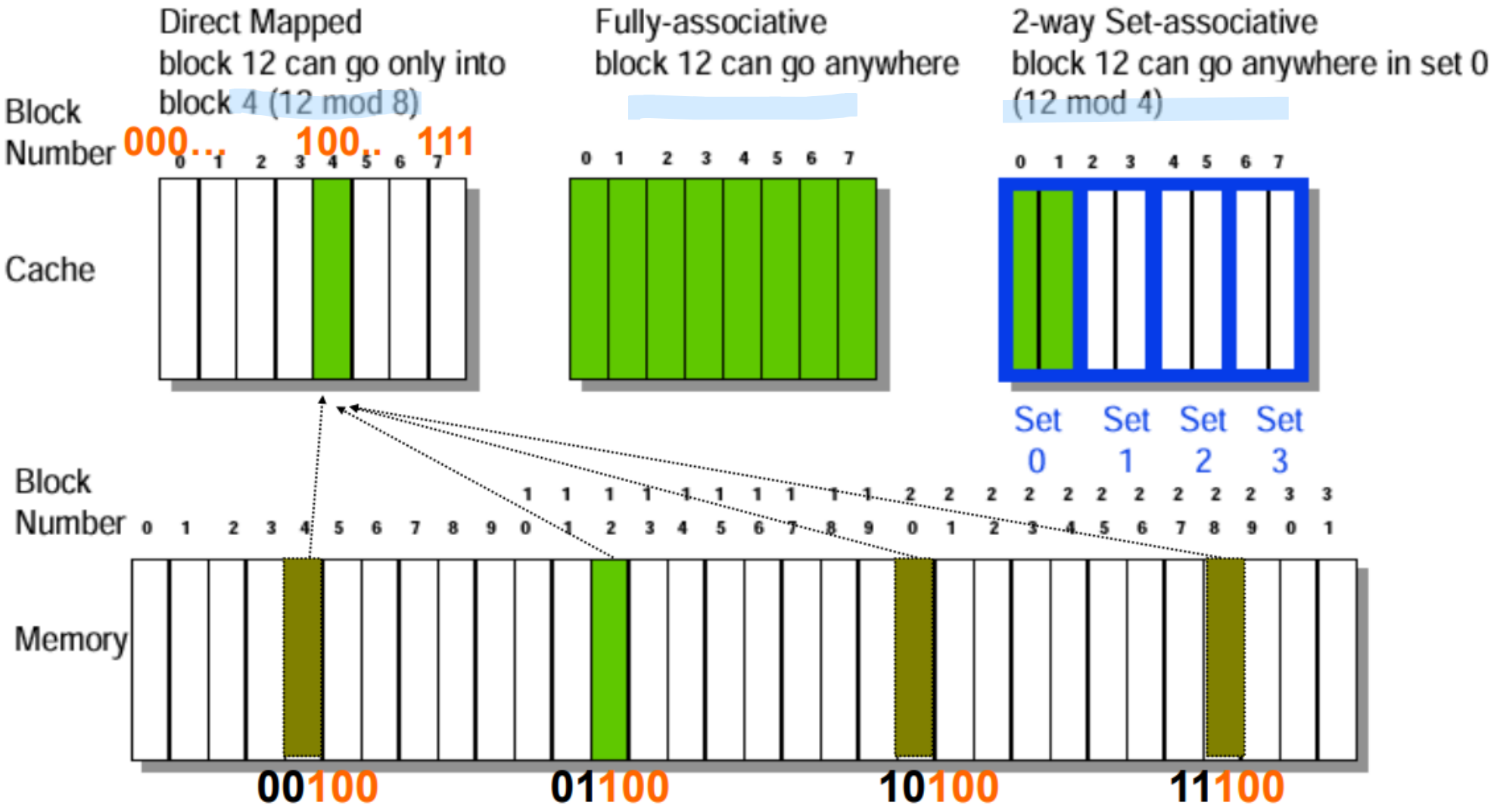
Where to place a Block (Line) of Data in Cache?

- Block Placement
  - Direct Mapped, Fully Associative, Set Associative
    - Direct mapped: (Block number) mod (Number of blocks in cache)
    - Set associative: (Block number) mod (Number of sets in cache)
      - # of set ≤ # of blocks
      - *n*-way: *n* blocks in a set
      - 1-way = direct mapped
    - Fully associative: # of set = 1



6

Cache Organization



Cache Hit, Miss and Replacement

What is Hit and Miss?

To study the Memory Hierarchy behaviour when a Read or Write Command discovers that the Data is available inside a Cache

Hit Policy for Write operation

- Write through
  - Update next level on every write → cache is always clean
  - Lots of traffic to next level (mostly writes)
- Write back
  - Write to cache and mark block dirty → update main memory on eviction
  - More complex eviction and coherence
- Reservation problem:
  - Reads use tag and data array at same time
  - Writes use tag first, then data array
  - How to pipeline to allow one read/write per cycle?

What Happens on a Write Command

Q4: What Happens on a Write?

	Write-Through	Write-Back
Policy	Data written to cache block, also written to lower-level memory	1. Write data only to the cache 2. Update lower level when a block falls out of the cache
Debug	Easy	Hard
Do read misses produce writes?	No	Yes
Do repeated writes make it to lower level?	Yes	No

Additional option -- let writes to an un-cached address allocate a new cache line ("write-allocate").

Write Hit Approach

- Write through
  - Update next level on every write → cache is always clean
  - Lots of traffic to next level (mostly writes)
- Write back
  - Write to cache and mark block dirty → update main memory on eviction (more complex eviction and coherence)
- Write buffer for write through
  - Processor: writes data into cache and write buffer
  - *Memory controller*: write contents of buffer to memory

